

MLFF-DATA9A6b Architecture and Observable-Evidence Consistency Specification

mdstats project

2026-07-30

Contents

1 Purpose	1
2 Ownership boundary	2
3 Analysis-owned recipe contract	2
3.1 Construction-time checks	2
3.2 Capability schema	2
3.3 Execution evidence	2
4 MLFF pairing and lineage	3
4.1 Collection identity	3
4.2 Candidate trajectory identity	3
4.3 Paired evidence	3
5 Profile terminology	3
6 Dependency-graph revision	4
7 Thermomechanical/energetic architecture gate	4
8 Tests	4
9 Next stage	4

1 Purpose

Supersession status. DATA9A6b is the historical consistency stage implemented in 0.20.45a0. DATA9A6c in 0.20.46a0 supersedes its lineage, result-identity, leakage-ordering, runtime-identity, and packaging contracts.

MLFF-DATA9A6b closes the consistency defects discovered after introducing the analysis-owned observable bridge. It does not add a new physical observable and does not move RDF, coordination, dynamics, transport, topology, or energetic algorithms into `mdstats.training_data`.

The stage has four responsibilities:

1. add construction-time dependency validation so observable recipes are dependency-safe before execution;
2. bind physical-validation evidence to the actual reference trajectory, candidate trajectory, model artifact, MD protocol, runtime, and analysis capability versions;
3. repair the MLFF architecture manual, stage plan, dependency graph, and documentation indices so that generic profiles are normative and LTA is an optional extension;
4. establish a separate thermomechanical and energetic validation architecture before any EOS, elasticity, viscosity, phonon, surface, interface, defect, or migration API is implemented.

2 Ownership boundary

`mdstats.analysis.observable_validation` owns only standardized dispatch and execution evidence. The owner analysis module retains the scientific theory, numerical method, units, uncertainty, and result type.

`mdstats.training_data.observable_validation` owns only:

- immutable reference/candidate pairing;
- candidate model and MD-generation lineage;
- material-profile recommendations;
- later comparison and checkpoint policies.

It must not normalize an RDF, select a diffusion plateau, integrate a stress correlation, fit an EOS, assemble force constants, or extract an NEB barrier.

3 Analysis-owned recipe contract

3.1 Construction-time checks

`ObservableAnalysisRecipe` shall reject:

- duplicate call IDs;
- unknown observable IDs;
- unsupported observable API versions;
- self-dependencies;
- unknown dependencies;
- forward dependencies;
- absent required dependency bindings;
- absent required parameters;
- failure of declared one-of parameter groups;
- parameters outside the registered capability schema.

An ordered recipe is therefore executable by construction with respect to its static dependency graph. Runtime input validity remains a separate preflight.

3.2 Capability schema

Every registered capability shall declare:

- stable observable ID and API version;
- owner module and owner architecture manual;
- collection requirements;
- required ordinary parameters;
- required dependency parameters;
- one-of parameter groups;
- supported argument names;
- versioned parameter-codec identity;
- result-type hint;
- callable-signature digest;
- immutable capability digest.

Collection requirements use semantic names such as `positions_and_cells`, `time_axis`, `velocities`, and `stresses`; they are not informal guesses at Python attribute names.

3.3 Execution evidence

Each executed call shall record:

- call and capability digests;
- owner module/API identity;
- warning messages;

- per-call runtime duration;
- result type;
- runtime package identity.

The result object remains the native owner-module result.

4 MLFF pairing and lineage

4.1 Collection identity

`ObservableCollectionIdentity` shall bind the analyzed frame selection and at least the following independent digests when available:

- geometry;
- dynamics/time axis;
- labels;
- provenance/source files.

A user label is descriptive metadata and is never a substitute for these digests.

4.2 Candidate trajectory identity

`MLFFTrajectoryGenerationIdentity` shall bind:

- model artifact digest;
- model manifest digest when present;
- MD protocol digest;
- engine and version;
- runtime environment digest;
- numerical precision policy;
- random seed when relevant.

The bridge shall fail closed when complete lineage is required but absent.

4.3 Paired evidence

`MLFFObservableValidationEvidence` shall include:

- plan and recipe digests;
- reference and candidate collection identities;
- candidate generation identity;
- per-call capability, warning, runtime, and result metadata;
- confirmation that both sides executed the same immutable recipe.

Scientific pass/degraded/fail classification is intentionally absent until DATA9A8 comparison policies are implemented.

5 Profile terminology

`ObservableRecommendationProfile` is the current flat advisory enum used only to suggest available observable IDs. `MaterialValidationProfile` remains a compatibility alias and must not be interpreted as the future general material profile system.

The future DATA9A7 profile is compositional and separately represents phases, geometry, chemistry modifiers, atom groups, condition axes, independence axes, and optional extensions. LTA/ring/site logic is one optional porous/zeolite extension.

6 Dependency-graph revision

At completion of DATA9A6b, the canonical graph was schema 10, architecture revision 5. DATA9A6c advances the current graph to schema 11, architecture revision 6. The DATA9A6b graph introduced generic nodes for:

- material-profile identity;
- atom-group, condition-axis, and independence-axis catalogs;
- partition and selection feature catalogs;
- profile event catalogs;
- observable recipes, collection identities, execution evidence, and comparison policy;
- thermomechanical/energetic validation recipes.

The LTA profile appears only as optional enrichment and must not be a mandatory predecessor of generic feature fitting or selection.

7 Thermomechanical/energetic architecture gate

The new architecture manual shall define theory, protocol identity, uncertainty, failure semantics, and staged implementation for:

- equation of state and equilibrium volume;
- elastic tensors and mechanical stability;
- thermal expansion, compressibility, and heat capacities;
- stress-autocorrelation shear and bulk viscosity;
- harmonic phonons and quasiharmonic thermodynamics;
- surface and interface energetics;
- neutral and externally corrected charged defects;
- minimum-energy paths and migration barriers;
- relative phase and formation energetics.

The manual shall explicitly defer structural distributions, displacement/current transport, topology statistics, and porous ring/site semantics to their existing owners.

8 Tests

The stage is accepted only when tests cover:

1. dependency-safe recipe construction and rejection paths;
2. machine-checkable collection preflight;
3. warning and runtime evidence;
4. collection/model/protocol lineage and tamper detection;
5. v1 recipe/plan read compatibility;
6. all registered dependency chains through the standardized facade;
7. dependency-graph generic/LTA-optional invariants;
8. documentation status, stage identifiers, and new architecture ownership;
9. source compilation, wheel build, and installed-wheel registry smoke.

9 Next stage

DATA9A7a introduces the real material-profile and atom-group contracts. DATA9A7b adds universal structural selection providers; DATA9A7c adds phase/geometry profiles; DATA9A7d migrates LTA behind porous/zeolite extensions; DATA9A7e qualifies multiple non-LTA and LTA systems. DATA9A8 then introduces observable comparison and acceptance policies before production DATA6–DATA8 evidence is regenerated.